

Tidy Data

Dr. Mutua Kilai, PhD

September-December 2026

Department of Pure and Applied Sciences

Tidy Data

Rules of tidy data

- i. Each variable must have its own column
- ii. Each observation must have its own row
- iii. Each value must have its own cell

Visual Display

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	1280425583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	1280425583

observations

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	1280425583

values

Pivoting data

- The principles of tidy data seem so obvious that you might wonder if you'll ever encounter a dataset that isn't tidy.
- Unfortunately, however, most data that you will encounter will be untidy. There are two main reasons:
 - i. Most people aren't familiar with the principles of tidy data, and it's hard to derive them yourself unless you spend a lot of time working with data.
 - ii. Data is often organised to facilitate some use other than analysis.

Solving pivoting issues

- Pivoting enables you to solve the following issues:
 - a. One variable might be spread across multiple columns.
 - b. One observation might be scattered across multiple rows.
- Typically a data set will only suffer from one of these problems.
- To fix these problems you will need to use `tidyr` package and the functions `pivot_longer()` and `pivot_wider()`

Data in Wider Format

- The example below denotes a data set in a wider format

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

- The data frame is not tidy because multiple observations of year appear in each row of the data frame (i.e., the single measurement variable, year, is dispersed across two columns).

Data in Longer Format

country	year	cases
Afghanistan	1999	745
Afghanistan	2000	2666
Brazil	1999	37737
Brazil	2000	80488
China	1999	212258
China	2000	213766

- The data frame is tidy because each row corresponds to one observation (i.e., a unique combination of country, year and cases) and the single measurement variable, year, now appears in only one column.

Pivot Longer `pivot_longer()`

- Pivoting to a longer format is usually needed if multiple observations of the same variable are recorded on individuals but are stored in separate columns of the data frame.
- To tidy a data set like this we need to **pivot** the offending columns into a new pair of variables

⚠ Requirements to `pivot_longer()`

- i. The set of column names whose names are values not variables. This is denoted using `cols=`
- ii. The name of the variable to move the column names to. This will be `names_to=`
- iii. The name of the variable to move the column values to. This will be `values_to=`

Example 1

- We use the inbuilt data in `tidyr` package named `table4a` which looks like this

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

- We need to change the data set from the wide format to long format.

Example 1

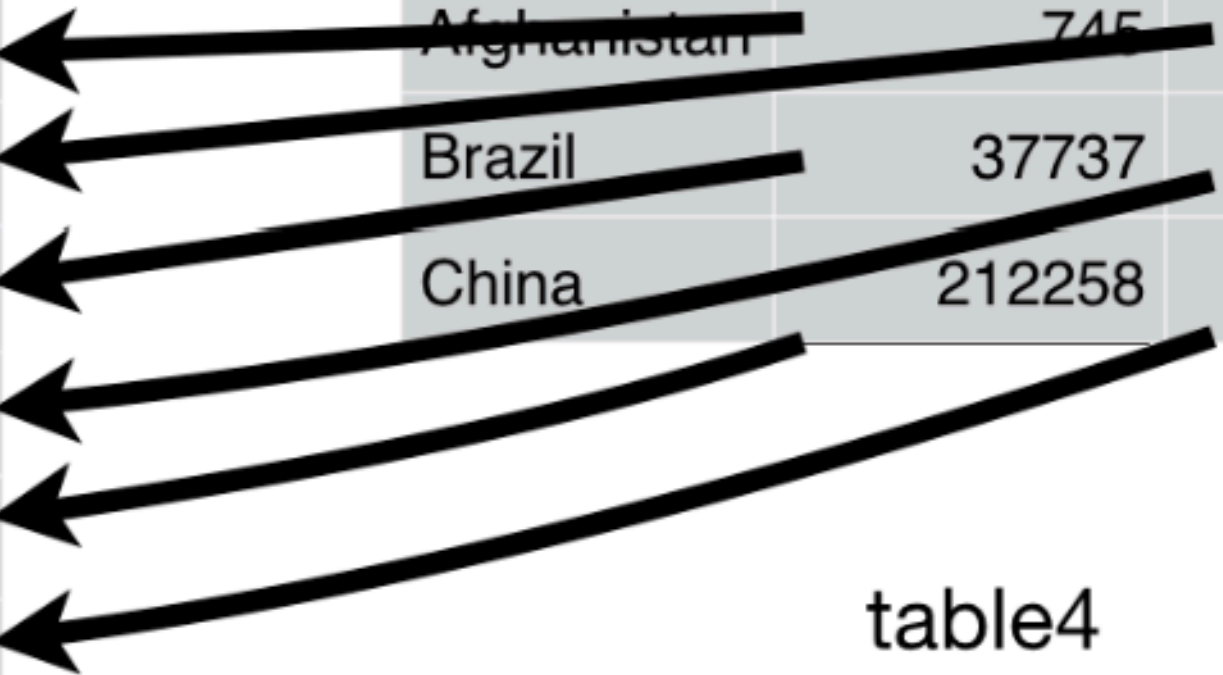
- We use the `pivot_longer()` function and specify the `cols`, `names_to`, `values_to` as follows

```
1 library(tidyr)
2 long <- table4a %>%
3   pivot_longer(c(`1999`, `2000`), names_to = "year", values_to = "cases")
4 knitr::kable(long)
```

country	year	cases
Afghanistan	1999	745
Afghanistan	2000	2666
Brazil	1999	37737
Brazil	2000	80488
China	1999	212258
China	2000	213766

Description

- `pivot_longer()` makes datasets longer by increasing the number of rows and decreasing the number of columns.



country	year	cases
Afghanistan	1999	745
Afghanistan	2000	2666
Brazil	1999	37737
Brazil	2000	80488
China	1999	212258
China	2000	213766

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

table4

Exercise

Convert the following data from wide format to long format

id	X1	X2
A	1	5
B	2	6
C	3	7
D	4	8

Pivot Wider `pivot_wider()`

- `pivot_wider()` is the opposite of `pivot_longer()`.
- You use it when an observation is scattered across multiple rows.

⚠ Requirements

- To pivot from a longer to a wider format you must:
 - i. Identify the variable in the longer format that contains the names of the new variables in the new wider format. This will be `names_from=`
 - ii. Identify the variable in the longer format that contains all of the values of the measurement variable. This will be `values_from=`

Example of Long data to convert to wide format

country	year	type	count
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362

Description

- An observation is a country in a year, but each observation is spread across two rows.
- To convert the data we need to specify where to extract the names from and where to get the values from

In R

```
1 library(tidyr)
2 wide <- table2 %>%
3   pivot_wider(names_from = type, values_from = count)
4 knitr::kable(head(wide))
```

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Visual Display

country	year	key	value
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898
China	1999	cases	212258
China	1999	population	1272915272
China	2000	cases	213766
China	2000	population	1280428583

table2

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Note

Note

`pivot_longer()` makes wide tables narrower and longer; `pivot_wider()` makes long tables shorter and wider.

Separating Variables

- Consider the data below

country	year	rate
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583

separate()

- `separate()` pulls apart one column into multiple columns, by splitting wherever a separator character appears.

```
1 sepa <- table3 %>%
2   separate(rate, into = c("cases", "population"), sep = "/")
3 knitr::kable(sepa)
```

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

unite

- `unite()` is the inverse of `separate()`: it combines multiple columns into a single column.
- We will unite the columns `century` and `year`

country	century	year	rate
Afghanistan	19	99	745/19987071
Afghanistan	20	00	2666/20595360
Brazil	19	99	37737/172006362
Brazil	20	00	80488/174504898
China	19	99	212258/1272915272
China	20	00	213766/1280428583

In R

```
1 wwu <- table5 %>%
2   unite(new, century, year, sep = "")
3 knitr::kable(wwu)
```

country	new	rate
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583